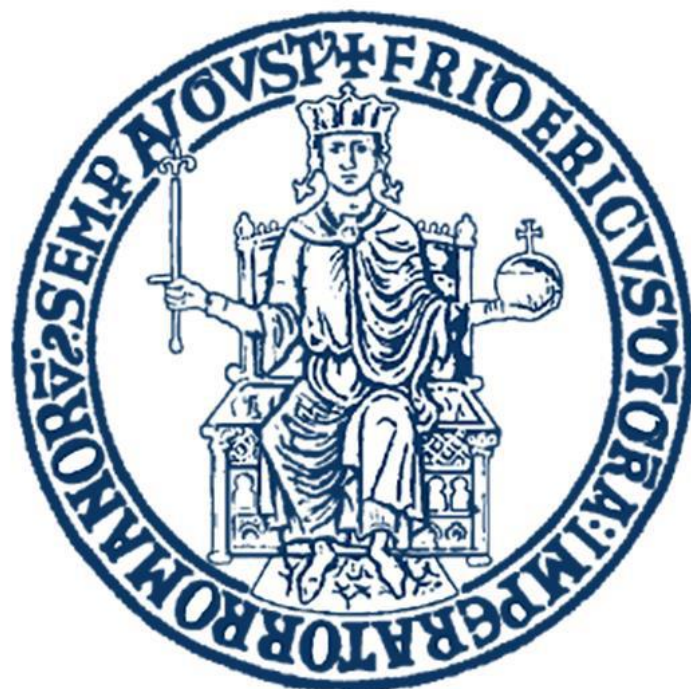




UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

DOCUMENTAZIONE PER PROGETTO
DI INGEGNERIA DEL SOFTWARE

CdL Triennale in Informatica

AGOSTINO SORRENTINO
N86005123
MARIATERESA PRINCIPATO
N86005284

ANNO ACCADEMICO: 2025/2026

SOMMARIO

1	Introduzione	3
2	Specifica dei Requisiti Software	3
2.1	Glossario	3
2.2	Modellazione tramite Use Case Diagram	5
2.2.1	UCD Generale	5
2.2.2	UCD Crea Account	5
2.2.3	UCD Crea Issue	6
2.2.4	UCD Autenticazione	6
2.2.5	UCD Visualizza/Gestisci Issues	6
2.3	Individuazione e Caratterizzazione del Target degli Utenti	7
2.4	Descrizione dei Requisiti Non-Funzionali e di Sistema	8
2.4.1	Requisiti di Sistema	8
2.4.2	Requisiti di Usabilità	8
2.4.3	Requisiti di Sicurezza	8
2.4.4	Requisiti di Manutenibilità e Scalabilità	9
2.5	Formalizzazione di casi d'uso significativi	9
2.5.1	Descrizione Testuale via Formalismo Tabellare Cockburn	9
2.5.2	Prototipazione Visuale via Mock-up (realizzato con Figma)	13
3	Documento di Design del Sistema	14
3.1	Architettura Proposta	14
3.2	Criteri di Design e Motivazioni	14
3.3	Scelte Tecnologiche	15
4	Attività di Testing	16
4.1	Test Plan per funzionalità a scelta	16
4.2	Codice per Test di Unità Automatici	16
4.3	Strategie di Testing	19

1 INTRODUZIONE

Il documento ha lo scopo di definire le specifiche dei requisiti software per il progetto BugBoard26. Esso descrive funzionalità, vincoli e interfacce fungendo da guida per sviluppo, testing e manutenzione dell'applicazione.

BugBoard26 è un sistema software di tracciamento dei problemi (issue) progettato per semplificare la gestione e la risoluzione di segnalazioni all'interno di un team di sviluppo o organizzazione. L'obiettivo è fornire una piattaforma dove gli utenti possono segnalare issue, assegnarli ai membri del team, monitorare lo stato e comunicare attraverso un sistema di notifiche.

Il codice, la documentazione completa, Javadoc e il file jar eseguibile è disponibile all'indirizzo <https://github.com/aaagoo/BugBoard26>.

2 SPECIFICA DEI REQUISITI SOFTWARE

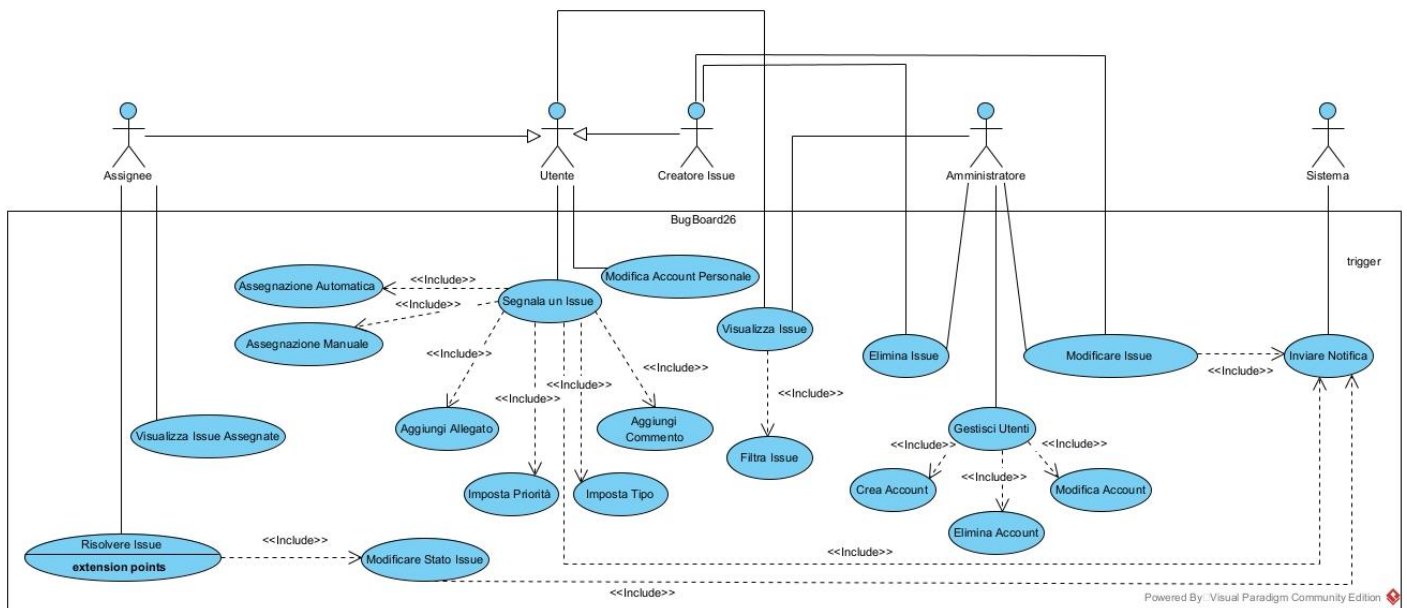
2.1 GLOSSARIO

Termine	Definizione
BugBoard26	Nome del sistema software di bug tracking
Issue	Rappresenta un problema, una richiesta di funzionalità, una domanda o un task di documentazione
Bug	Tipo di Issue che segnala un errore o un malfunzionamento nel software
Feature	Tipo di Issue che richiede l'implementazione di nuove funzionalità
Question	Tipo di Issue che segnala una domanda o chiarimento in riferimento a qualcosa
Documentation	Tipo di Issue che richiede la scrittura di documentazione
Account	Utente registrato nel sistema, identificato da username/email e password
Amministratore	Utente con privilegi elevati: può gestire gli account (creare, modificare, eliminare), visualizzare tutte le Issue e intervenire su di esse

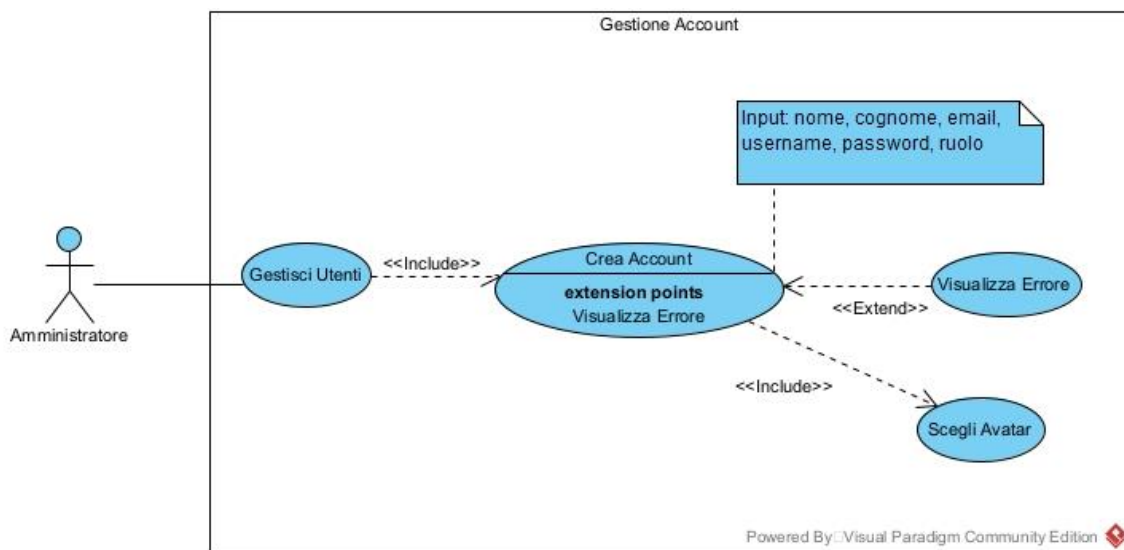
Utente (standard)	Utente base: può creare Issue, visualizzare tutte le Issue ma modificarle/risolverle solo se ne è il creatore/assegnatario
Assegnatario	L'utente incaricato di risolvere una specifica Issue
Creatore	L'utente che ha aperto (segnalato) una Issue
Stato (Risolto)	Condizione di una Issue. Se "Risolto" è vero, il lavoro è completato
Priorità	Livello di urgenza di una Issue (LOW, MEDIUM, HIGH, CRITICAL)
Notifica	Messaggio di sistema inviato a un utente per informarlo di eventi rilevanti (es. assegnazione di una issue, risoluzione di issue creata)
Dashboard	Schermata che mostra l'elenco delle issue e permette di filtrarle e gestirle
Backend	La parte del sistema che gira su server (Spring Boot), gestisce la logica di business e comunica con il database
Frontend (Client)	L'interfaccia grafica (Java Swing) con cui l'utente interagisce
API REST	Interfaccia di comunicazione tra Frontend e Backend basata su protocollo HTTP e formato JSON

2.2 MODELLAZIONE TRAMITE USE CASE DIAGRAM

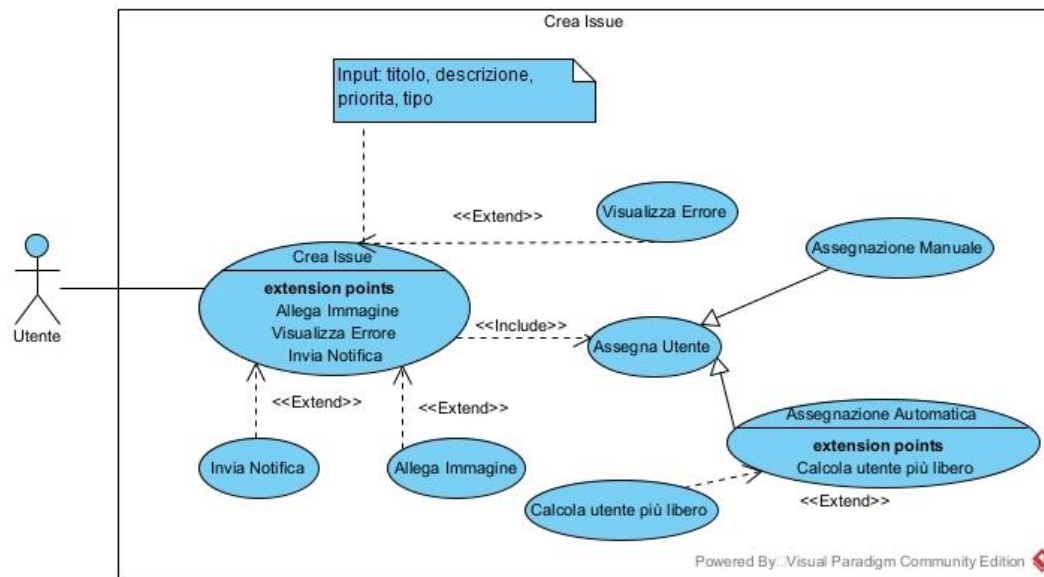
2.2.1 UCD Generale



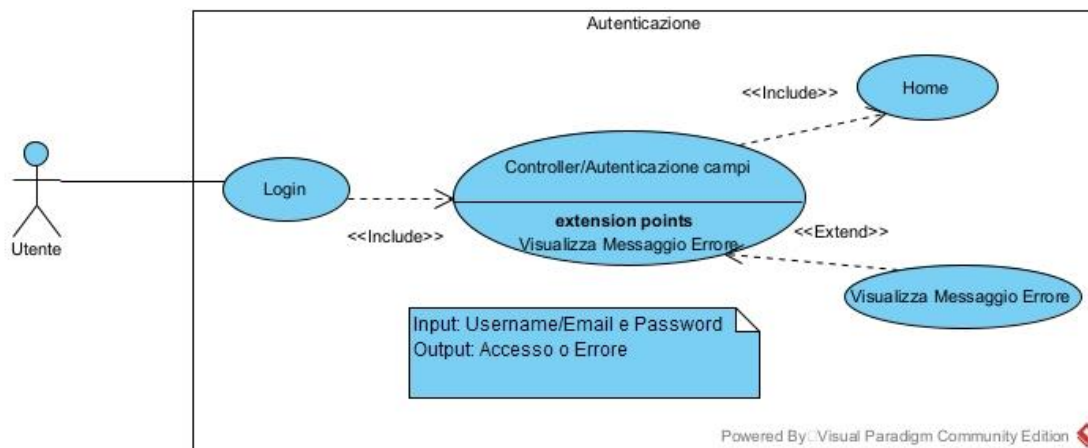
2.2.2 UCD Crea Account



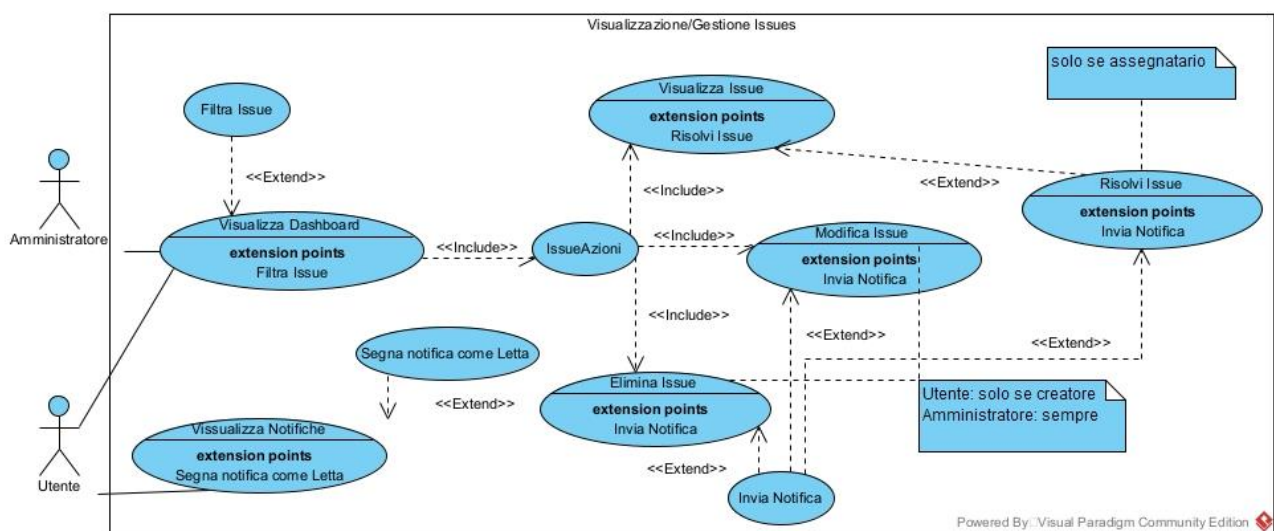
2.2.3 UCD Crea Issue



2.2.4 UCD Autenticazione



2.2.5 UCD Visualizza/Gestisci Issues



2.3 INDIVIDUAZIONE E CARATTERIZZAZIONE DEL TARGET DEGLI UTENTI

BugBoard26 è progettato per essere utilizzato all'interno di un team di sviluppo software o organizzazioni che necessitano di tracciare e gestire problemi tecnici.

Gli utenti del sistema si dividono in due categorie principali, caratterizzate da ruoli e obiettivi diversi.

1) Utente Standard (Developer / Tester / Reporter)

Composto da sviluppatori software, tester o personale incaricato di segnalare o risolvere problemi. Possiede competenze tecniche medio-alte e ha familiarità con il concetto di Issues.

Obiettivi Principali:

- segnalare nuovi bug o richieste.
- visualizzare le issue assegnate a se stesso.
- aggiornare lo stato delle issue per comunicare il progresso
- rimanere aggiornato tramite notifiche sulle issue

2) Amministratore (Project Manager / Team Lead)

I responsabili del progetto o il capo del team. Possiede competenze tecniche alte e conosce la struttura del team e le policy di gestione.

Obiettivi Principali:

- gestire l'accesso al sistema (gestire gli account)
- supervisionare il flusso di lavoro e intervenire in caso di errori (accesso completo alle issue e alla loro modifica/eliminazione)
- mantenere l'ordine del database per utenti e issue

2.4 DESCRIZIONE DEI REQUISITI NON-FUNZIONALI E DI SISTEMA

2.4.1 Requisiti di Sistema

Per garantire il corretto funzionamento di BugBoard26, sono richiesti i seguenti requisiti minimi:

Lato Server (Backend):

- Sistema Operativo: Qualsiasi OS che supporti Java (Linux, Windows, macOS)
- Java Runtime Environment (JRE): Versione 21 o superiore
- Connettività: connessione di rete stabile per comunicare con il Database e i client

Lato Client (Frontend):

- Sistema Operativo: Windows, macOS o Linux con supporto grafico
- Java Runtime Environment (JRE): Versione 21 o superiore
- Risoluzione Schermo: consigliata 1920 x 1080 per una visualizzazione ottimale. Il minimo richiesto è qualsiasi risoluzione superiore a 1200 x 800 (la dimensione delle schermate dell'applicazione)

2.4.2 Requisiti di Usabilità

L'interfaccia grafica deve usare il tema FlatLaf per un aspetto moderno e pulito. Inoltre, l'interfaccia non deve mai "congelarsi" durante le operazioni di rete. Tutte le chiamate al server devono essere eseguiti in thread separati mostrando indicatori di caricamento. Il sistema deve fornire feedback immediato per ogni azione.

2.4.3 Requisiti di Sicurezza

L'accesso al sistema è protetto dal login con username/email e password.

Gestione Sessione: Utilizzo di Token di Autenticazione (Bearer Token) per autenticare le chiamate API, evitando di trasmettere le credenziali ad ogni richiesta

Integrità dei Dati:

- Vincoli di chiave esterna nel database per prevenire dati orfani (es. issue senza creatore)

- Trigger e funzioni SQL per garantire regole di business (es. impossibile eliminare un amministratore se è l'unico amministratore nel sistema)

2.4.4 Requisiti di Manutenibilità e Scalabilità

- Architettura MVC: Separazione tra Modello (dati), Vista (GUI) e Controller (logica) per facilitare modifiche future
- Separazione Client-Server: Il backend espone API REST standard, permettendo in futuro di sviluppare altri client (es. web o mobile) senza modificare il server
- Codice Modulare: Utilizzo di classi di utilità e client specifici per evitare duplicazione di codice

2.5 FORMALIZZAZIONE DI CASI D'USO SIGNIFICATIVI

Abbiamo scelto di rappresentare “Crea Nuova Issue” e “Risolvi Issue”.

2.5.1 Descrizione Testuale via Formalismo Tabellare Cockburn

1) Crea Nuova Issue

USE CASE #1	Crea Nuova Issue
Goal in Context	Inserire con successo una nuova issue
Preconditions	L'utente deve essere collegato a internet e correttamente autenticato nel sistema
Success End Condition	Una nuova issue è inserita nel database con un ID univoco e stato iniziale “Risolto” impostato a “false”. Le informazioni della issue con eventuale allegato immagine sono salvati correttamente e l'utente è riportato alla schermata home dell'applicazione
Failed End Condition	Il salvataggio fallisce per mancanza di dati inseriti (mancato titolo, descrizione, priorità, tipo) e viene mostrato un errore. Il database non salva la issue problematica.
Primary Actor	Utente Standard
Trigger	L'utente clicca sul pulsante “Nuova Issue” e si apre la schermata relativa alla creazione

Main Scenario	Step n.	Actor 1	Actor 2	System
	1	Clicca sul pulsante “Nuova Issue” dalla Home		
	2		Il sistema mostra il form relativo alla creazione dell’issue	
	3	L’utente inserisce i dati relativi alla issue e clicca “Conferma”		
	4		Il sistema valida i dati, salva la issue e mostra un messaggio di successo	
	5		Il sistema reindirizza l’utente alla schermata Home	
Extension A: Dati Mancanti	4a		Il sistema nota che l’utente non ha inserito dati in campi obbligatori: viene mostrato un errore ed è impedito il salvataggio	
Extension B: Allegato Troppo Grande	4b		Il sistema nota che l’allegato della issue supera i 2MB: viene mostrato un errore ed è impedito il salvataggio	

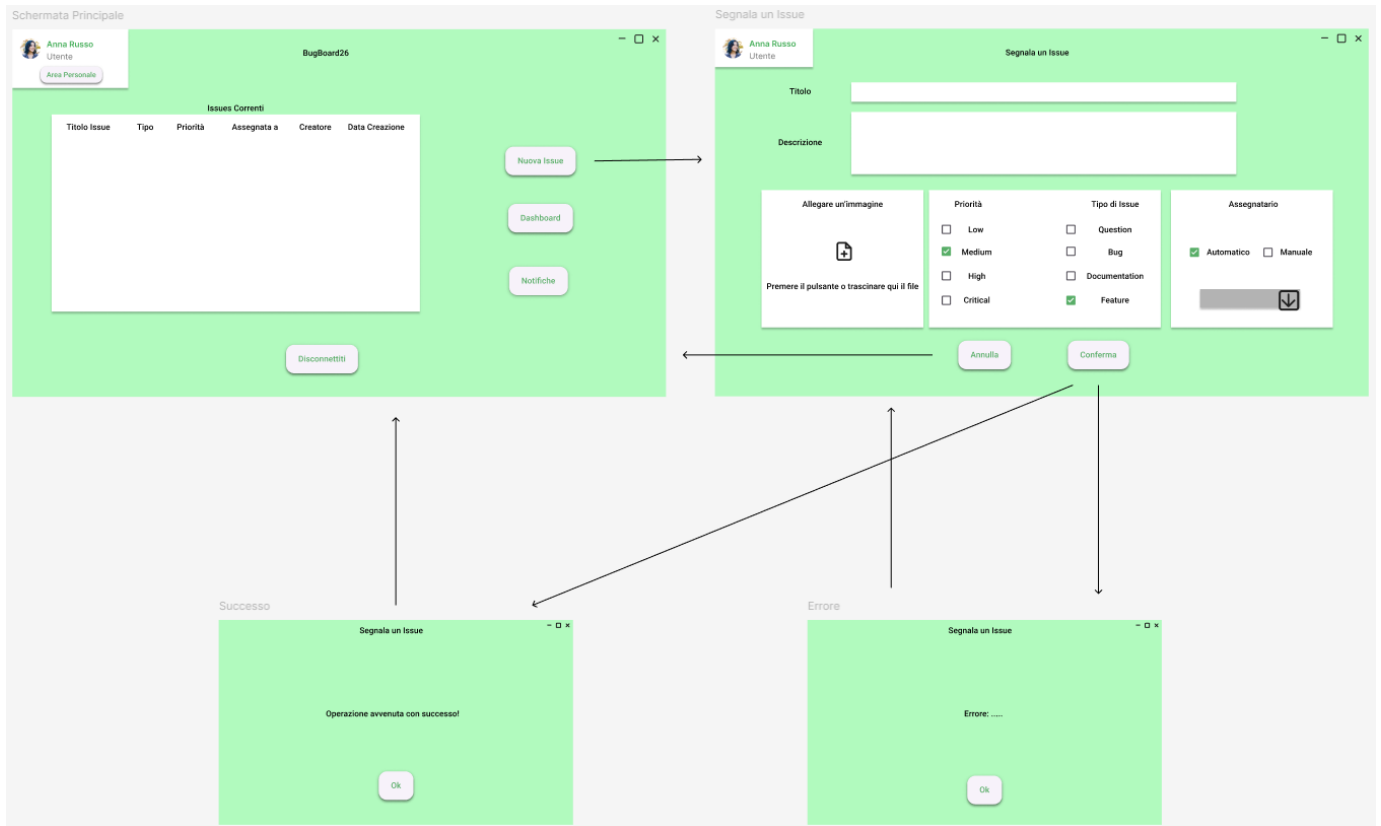
2) Risolvi Issue

USE CASE #2	Risolvi Issue			
Goal in Context	Risolvere una issue assegnata all'utente			
Preconditions	L'utente deve essere autenticato e deve essere l'assegnatario della issue. La issue deve essere nello stato "Risolto" impostato come "false"			
Success Condition	End	L'issue sarà impostata nel campo "Risolto" come "true", la data di risoluzione viene registrata e il creatore riceve una notifica		
Failed Condition	End	Lo stato "Risolto" della issue non cambierà		
Primary Actor	Utente Assegnatario			
Trigger	L'utente, che al momento si trova in VisualizzaIssue clicca il pulsante "Modifica Stato Issue"			
Main Scenario	Step n.	Actor 1	Actor 2	System
	1	L'utente visualizza la Dashboard o la Home		
	2	L'utente clicca su una issue assegnata a lui		
	3		Il sistema apre la issue con VisualizzaIssue e rende il pulsante "Modifica Stato Issue" cliccabile	
	4	L'utente clicca sul pulsante "Modifica Stato Issue"		
	5		Il sistema apre un pop up chiedendo conferma	
	6	L'utente conferma		

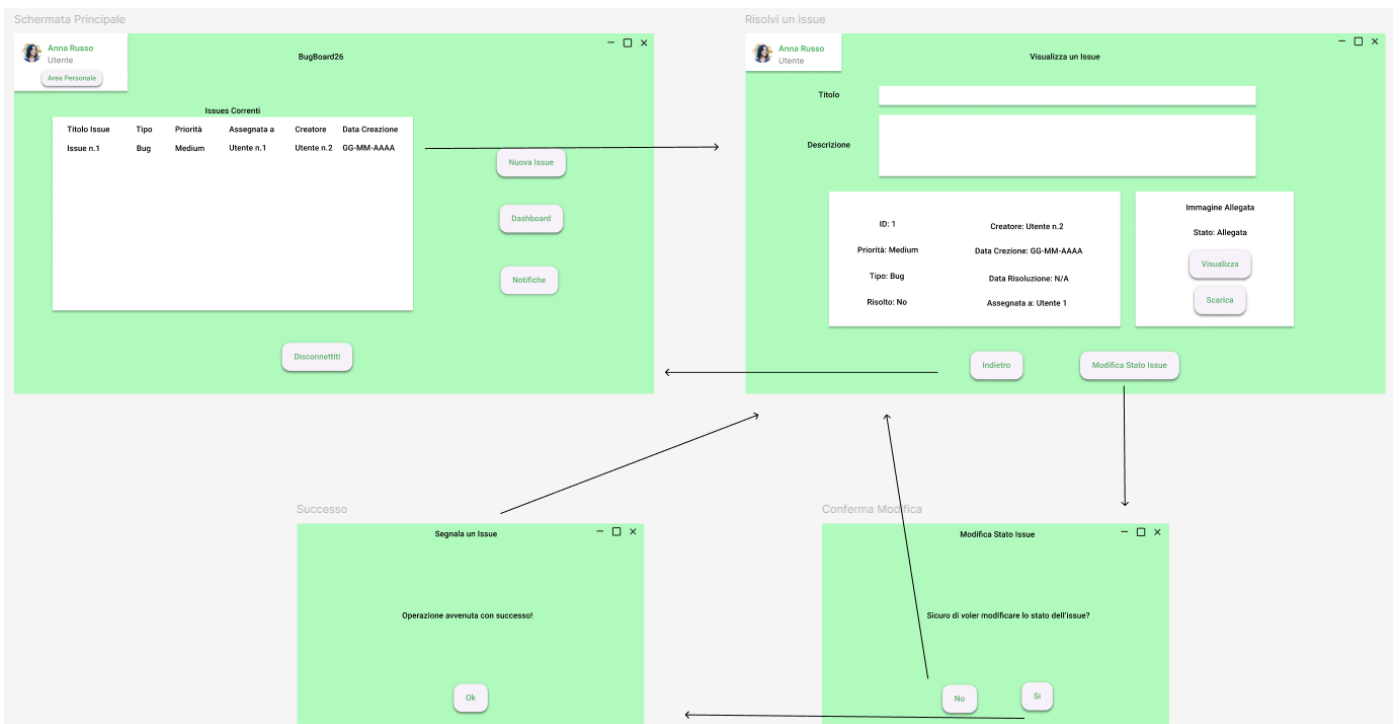
	7		Il sistema aggiorna lo stato nel Database e invia la notifica	
	8		La GUI si aggiorna mostrando lo stato "Risolto" a "si". Nel database invece sarà impostato a "true"	
Extension A: L'Utente non è l'Assegnatario	2a	L'utente clicca su una issue non assegnata a lui		
	3a		Il sistema apre la issue con VisualizzaIssue, ma l'issue non è assegnata a lui, quindi il pulsante "Modifica Stato Issue" non è cliccabile	

2.5.2 Prototipazione Visuale via Mock-up (realizzato con Figma)

1) Crea Nuova Issue



2) Risolvi Issue



3 DOCUMENTO DI DESIGN DEL SISTEMA

3.1 ARCHITETTURA PROPOSTA

L'architettura del sistema BugBoard26 è stata progettata seguendo i principi di progettazione Object-Oriented, garantendo che la logica applicativa sia completamente gestita dal codice sviluppato senza delegare nulla a servizi esterni. Il sistema adotta un'architettura client-server a tre livelli basata sul pattern MVC (Mode-View-Controller), dove ogni componente ha responsabilità definite ed è indipendente dalla struttura sottostante:

- Presentation Layer (Client – Frontend):

Realizzato in Java Swing UI Designer, è responsabile dell'interazione con l'utente, della visualizzazione dei dati e dell'inserimento degli input. Comunica con il Business Layer tramite un'interfaccia API REST, ignorando l'esistenza del database.

- Business Logic Layer (Server – Backend):

Realizzato con Spring Boot, implementa le regole di business in codice Java. Inoltre, gestisce la persistenza dei dati. Sebbene utilizzi Supabase come storage fisico, l'accesso ai dati è gestito da un layer di servizio e repository, rendendo il sistema portabile su qualsiasi altro database relazionale con modifiche minime al codice. Il server non si limita a inoltrare richieste al Database, ma elabora, trasforma e valida i dati seguendo il modello Object-Oriented.

- Data Layer (Database Cloud):

Utilizzato esclusivamente come Relational Database Management System. Non vengono utilizzate le funzionalità di Supabase MBaaS che bloccherebbero il progetto a questo specifico Database. Esso è trattato come un semplice contenitore di dati relazionali, integrato tramite linguaggio SQL, in modo da rispettare le richieste del contraente.

3.2 CRITERI DI DESIGN E MOTIVAZIONI

Il design è guidato dalle classi di dominio definite nel codice Java, non dalle tabelle nel database. Questo assicura che la logica sia manutenibile.

Ogni livello conosce solo il livello immediatamente sottostante. Il Client non sa del Database, il Database non sa della GUI: questo favorisce la testabilità e la scalabilità del software.

La scelta di usare Supabase solo come host PostgreSQL, gestendo l'autenticazione e la logica nel backend Java, risponde alla necessità di non dipendere da servizi MBaaS (Mobile Backend as a Service) per le funzionalità core. Ciò significa che il sistema è autonomo.

3.3 SCELTE TECNOLOGICHE

Tecnologia	Ruolo	Motivazione della Scelta
Java 21	Linguaggio di Programmazione Core	Scelto per la sua robustezza, la portabilità e il supporto alla programmazione Object-Oriented. La versione 21 garantisce stabilità e compatibilità completa con le altre parti del progetto
Spring Boot 3	Framework Backend	Utilizzato per implementare il Business Logic Layer. Offre container per la gestione delle dipendenze e l'esposizione di API REST.
PostgreSQL (via Supabase)	Database Relazionale	Scelto come motore RDBMS per la persistenza. L'hosting su Supabase è usato solo per vantaggi logistici (accessibilità cloud), ma serve solo come Database SQL standard, senza sfruttare le API della piattaforma
Java Swing UI Designer	Framework GUI	Libreria standard e nativa per interfacce desktop Java.
FlatLaf	Libreria UI	Il classico look di Swing è stato modificato tramite l'utilizzo di FlatLaf per migliorarlo e stare al passo con le applicazioni desktop moderne
Jackson	Data Binding	Usato per disaccoppiare Client e Server. Permette di trasformare gli oggetti del dominio in formato JSON neutro per la trasmissione via rete
Maven	Project Manager	Garantisce la gestione delle dipendenze e del ciclo di vita del software

4 ATTIVITÀ DI TESTING

4.1 TEST PLAN PER FUNZIONALITÀ A SCELTA

Creazione Nuova Issue: verifica che un utente autenticato possa creare correttamente una nuova issue nel sistema.

Ambiente di Test:

- Client: Windows 11, Java 21
- Server: Locale
- Database: Supabase

Test Case

ID	Descrizione	Input	Risultato Atteso
TC1	Creazione con dati validi	Titolo: "Bug Login", Descrizione: "Errore 500", Priorità: "HIGH", Tipo: "BUG"	Messaggio "Issue creata con successo e assegnata a...". La issue appare nella Dashboard
TC2	Creazione con campi mancanti	Titolo: "", Descrizione: "Errore 500", Priorità: "HIGH", Tipo: "BUG"	Messaggio di errore "Inserisci un titolo". Nessuna issue creata
TC3	Creazione con allegato valido	... + File .png o .jpeg minore di 2MB	Issue creata, immagine memorizzata
TC4	Creazione con allegato non valido	... + File diverso dal formato .png o .jpeg o maggiore di 2MB	Messaggio di errore "File troppo grande" o "Formato file non valido"
TC5	Assegnazione Manuale	--- + Assegnatario: "nomeUtente"	Issue creata e assegnata a "nomeUtente". Notifica inviata a nomeUtente

4.2 CODICE PER TEST DI UNITÀ AUTOMATICI

Abbiamo scelto di testare due metodi della logica di business nel backend: "AccountService.login" e "IssueService.creaIssue".

1) AccountService (login)

```
@ExtendWith(MockitoExtension.class)
class AccountServiceTest {

    @Mock
    private JdbcTemplate jdbcTemplate;

    @InjectMocks
    private AccountService accountService;

    @Test
    void testLoginSuccesso() {
        //Preparazione dei dati e del comportamento simulato del DB
        String username = "admin";
        String password = "password";

        //Simula la risposta della funzione SQL 'login'
        when(jdbcTemplate.queryForMap(contains("SELECT * FROM login"), eq(username), eq(password)))
            .thenReturn(Map.of("successo", true));

        //Simula la risposta della query per recuperare i dati utente
        Map<String, Object> mockUserData = Map.of(
            "nomeutente", "admin",
            "password", "password",
            "nome", "Mario",
            "cognome", "Rossi",
            "email", "mario@test.com",
            "ruolo", "AMMINISTRATORE",
            "avatar", "user.png",
            "issueassegnate", 0
        );
        when(jdbcTemplate.queryForMap(contains("SELECT nomeUtente"), eq(username), eq(username)))
            .thenReturn(mockUserData);

        //Esecuzione del metodo da testare
        Account result = accountService.login(username, password);

        //Verifica dei risultati
        assertNotNull(result, "L'account restituito non deve essere null");
        assertEquals("admin", result.getNomeUtente());
        assertEquals(Ruolo.AMMINISTRATORE, result.getRuolo());
    }

    @Test
    void testLoginFallito() {
        // Arrange
        String username = "wrong";
        String password = "wrong";

        // Simula login fallito dal DB
        when(jdbcTemplate.queryForMap(contains("SELECT * FROM login"), eq(username), eq(password)))
            .thenReturn(Map.of("successo", false));

        // Act
        Account result = accountService.login(username, password);

        // Assert
        assertNull(result, "Il login con credenziali errate deve restituire null");
    }
}
```

2) IssueService (crealIssue)

```
ExtendWith(MockitoExtension.class)
class IssueServiceTest {
```

```
    @Mock
    private JdbcTemplate jdbcTemplate;
    @InjectMocks
    private IssueService issueService;
```

```
    @Test
    void testCreaIssueSuccesso() {
```

```
        // Arrange
        String titolo = "Bug Login";
        String descrizione = "Errore 500";
        Priorita priorita = Priorita.HIGH;
        Tipo tipo = Tipo.BUG;
        String creatore = "user1";
        String assegnatario = "dev1";
        String immagine = "url_img";
```

```
        // Simula la chiamata alla funzione SQL 'crea_issue'
        when(jdbcTemplate.queryForMap(
            contains("SELECT * FROM crea_issue"),
            eq(titolo),
            eq(descrizione),
            eq("HIGH"),
            eq("BUG"),
            eq(creatore),
            eq(assegnatario),
            eq(immagine)
        )).thenReturn(Map.of("messaggio", "Issue creata con successo"));
```

```
        // Act
        String risultato = issueService.creaIssue(titolo, descrizione, priorita, tipo, creatore, assegnatario, immagine);
```

```
        // Assert
        assertTrue(risultato.contains("Issue creata con successo"));
```

```
    }
```

```
    @Test
    void testCreaIssueFallito() {
```

```
        // Arrange
        String titolo = "Titolo Errato";
        String descrizione = "Descrizione";
        Priorita priorita = Priorita.LOW;
        Tipo tipo = Tipo.QUESTION;
        String creatore = "user1";
        String assegnatario = "dev1";
        String immagine = null;
        String messaggioErroreDB = "Errore: Titolo troppo lungo";
```

```
        // Simula la chiamata SQL che restituisce un messaggio di errore
        when(jdbcTemplate.queryForMap(
            contains("SELECT * FROM crea_issue"),
            eq(titolo), eq(descrizione), eq("LOW"), eq("QUESTION"),
            eq(creatore), eq(assegnatario), eq(immagine)
        )).thenReturn(Map.of("messaggio", messaggioErroreDB));
```

```
        // Act
        String risultato = issueService.creaIssue(titolo, descrizione, priorita, tipo, creatore, assegnatario, immagine);
```

```
        // Assert
        assertTrue(risultato.contains("Errore: Titolo troppo lungo"), "Il risultato dovrebbe contenere il messaggio di errore del DB");
    }
}
```

4.3 STRATEGIE DI TESTING

Per la verifica del sistema BugBoard26, è stata adottata una strategia mista che combina test automatici di unità per la logica backend e test manuali di sistema per l'interfaccia utente.

L'obiettivo è stato isolare e verificare la logica di business dal Backend indipendentemente dalle dipendenze esterne (Database e Rete).

Per l'esecuzione dei test e le asserzioni abbiamo utilizzato JUnit 5, mentre per la simulazione delle dipendenze abbiamo fatto uso di Mockito. Con esso abbiamo infatti creato oggetti "mock" del database poichè i servizi dipendono fortemente dal database JdbcTemplate. Ciò ci ha permesso di:

- simulare risposte specifiche del Database;
- verificare che il Service gestisca correttamente questi scenari senza dover realmente connettersi a Supabase
- garantire l'esecuzione rapida dei test (dato che non dobbiamo dare conto alla latenza di rete).

Per il metodo login, in particolare, abbiamo scritto due test distinti:

- "testLoginSuccesso" simula una risposta positiva del Database e verifica che venga restituito un oggetto Account valido;
- "testLoginFallito" simula una risposta negativa e verifica che il metodo restituisca "null"

Per il metodo creaIssue abbiamo applicato la stessa logica:

- "testCreaIssueSuccesso" verifica che, dato un inserimento corretto nel DB, il service restituisca un messaggio di conferma;
- "testCreaIssueFallito" simula un errore nel Database (es. violazione di vincoli o dati non validi) e verifica che il service propaghi correttamente l'errore

Per ridurre il numero di casi di test mantenendo una buona copertura, abbiamo raggruppato gli input in classi di equivalenza: in "login" abbiamo considerato tutti gli username/password errati come un'unica classe "Credenziali Invalide" scrivendo un solo test rappresentativo invece di testare infinite combinazioni errate; in "creazioneIssue" per l'upload abbiamo usato due classi per gestire la dimensione dei file (accettati se minori di 2MB, rifiutati altrimenti).

5 MANUALE DELL'UTENTE

5.1 REQUISITI DI SISTEMA

Per eseguire BugBoard26 è necessario disporre di:

- Sistema Operativo: Windows, macOS, Linux;
- Javat Runtime Environment (JRE): Versione 21 o superiore;
- Connessione Internet attiva

5.2 INSTALLAZIONE E AVVIO

L'applicazione è distrubuita come un singolo file eseguibile "BugBoard26.java". Non è richiesta alcuna installazione. E' richiesto solo di scaricare il file .jar e avviarlo con un doppio click.

Il sistema è preimpostato con un account Amministratore già esistente con dati:

- username: admin
- password: admin.

Una volta eseguito l'accesso al sistema, è possibile creare nuove utenze tramite questo account (o altri account Amministratore).